

Estimating LLM Inference Memory Requirements

Memory & GPU planning for large-language-model inference

Why Memory Matters for LLM Inference

In inference mode (not training), understanding memory consumption is critical for successful deployment. Under-estimating required memory can lead to OOM errors or inefficient resource allocation.

Model Weights

Parameters stored in GPU memory

Activation & Hidden States

Intermediate layer outputs maintained during forward pass computation

KV-Cache

Key-value tensors for autoregressive generation, growing with sequence length

Parallelism Overheads

GPU tensor slicing and distributed inference introduce duplicated buffers

Key Factors Influencing Memory Usage

Multiple dimensions affect the total memory footprint of your LLM deployment. Context length, model architecture, and parallelism strategies all compound to determine your GPU requirements.



Model Size

Parameter count directly impacts weight storage and activation size across layers



Context Length

Longer sequences exponentially increase KV-cache memory requirements



Batch Size

Parallel sequence processing multiplies memory usage linearly



GPU Parallelism

Tensor and pipeline parallel strategies introduce overhead and buffer duplication

Memory Breakdown Components

Understanding each memory component helps you accurately estimate total GPU requirements and identify optimization opportunities.

1

Model Weights

Number of parameters $\times$ bytes per parameter (e.g., 2 bytes for FP16, 1 byte for INT8)

2

Activation Buffers

Intermediate layer outputs stored during forward pass computation (estimate 25% of model weights size)

3

KV-Cache Storage

Key and value tensors for each layer, token, and generation step. This is the big one.

4

Framework Overheads

Duplicated buffers from parallelism and framework-specific memory allocations

Example Calculation: Model Weights and Activation Buffers

Let's walk through a concrete example to estimate weight memory for a 10 billion parameter model.

Assumptions

- Model size: 10 billion parameters
- Precision: 16-bit floats (FP16)
- Storage: 2 bytes per parameter

Calculation

10 billion parameters $\times$ 2 bytes = **20 GB** for model weights

Activation Buffers = $.25 \times 20\text{GB} = 4\text{GB}$

Total = 24 GB

Optimization: Quantization

Using 8-bit quantization (INT8) reduces weight storage:

10 billion parameters $\times$ 1 byte = **10 GB**

Activation Buffers = $.25 \times 10\text{GB} = 2\text{GB}$

Total = 12 GB

50% memory reduction with minimal accuracy impact for many use cases

Example Calculation: KV-Cache Memory

The KV-cache often dominates memory usage for long-context inference. Here's how to estimate it.

01

Define Architecture Parameters

Context length: 2048 tokens | Model layers: 40
| Hidden size: 4096 | Precision: 4 bytes (FP32)

02

Calculate Per-Token Storage

Each layer stores key + value tensors (the 2 below).

$$= 2 * \text{Precision} * \text{Layers} * \text{Hidden Dimension}$$

Simplified: $2 \times 4 \times 40 \times 4096$ bytes

~1.3 MB per token per layer

03

Scale Across Context & Layers

$$\text{Total KV Cache} = \text{KV Cache per Token} \times \text{Context Length} \times \text{Concurrent Requests}$$

1.3 MB x 2048 tokens MB $\approx$ 2662.4 MB $\approx$ **2.7 GB** for KV-cache alone

 This calculation is illustrative. Actual KV-cache size depends on your specific architecture, attention mechanism, and optimization techniques like flash attention, multi-query attention or grouped-query attention.

Total Memory Estimate

Combining all components gives us the complete memory footprint for our example 10B parameter model.

20

2.7

4

26.7

Model Weights (GB)

KV-Cache (GB)

Activation +
Overhead (GB)

Total Estimate (GB)

2048 token context length

FP16 precision storage

Buffers and framework costs

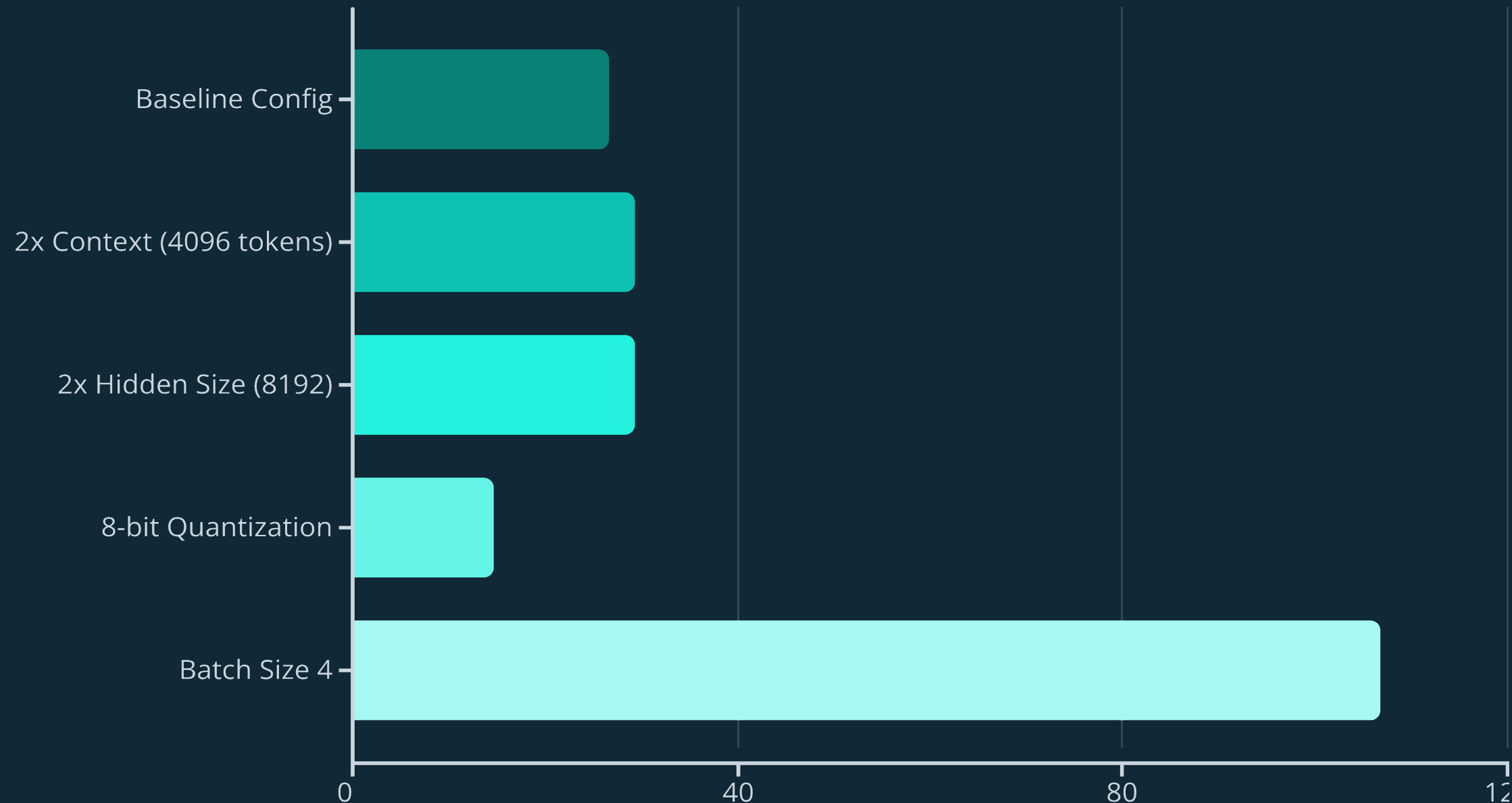
Baseline memory
requirement

GPU Selection: You'll need a GPU with memory safely above 27 GB for this configuration. [Add 10-20% safety margin](#) for system overhead and memory fragmentation.

Recommended: 40GB A100 or 48GB A6000 for this workload with comfortable headroom.

Impact of Varying Parameters

Small changes to key parameters can dramatically affect memory requirements. Understanding these relationships helps you right-size your infrastructure.



Key Insights: Doubling context length or hidden dimensions adds ~3 GB. Quantization provides immediate 45% savings. Batch size scales linearly with KV-cache and activations.

Summary

Proper memory planning is essential for successful LLM inference deployment. Use these calculations as a foundation for your infrastructure.

Memory Components Dominate

Model weights, activations, and KV-cache are the primary consumers. KV-cache may exceed weight memory for long contexts.

Calculate Before Deploying

Use simple calculations (demonstrated here) to estimate memory requirements ahead of time. Avoid costly OOM errors in production.

Key Optimization Levers

Model size, context length, batch size, precision, and parallelism strategy all compound to determine total memory footprint.

Plan for Growth

Always include buffer margin (10-20%) for overhead, longer contexts, larger models, and future scaling needs.

Next Steps: Apply these formulas to your specific model architecture. Test with profiling tools. Iterate on quantization and parallelism strategies to find the optimal memory-performance balance.